



UNIVERSIDADE
FEDERAL DO CEARÁ



Índices Estruturados em Árvores

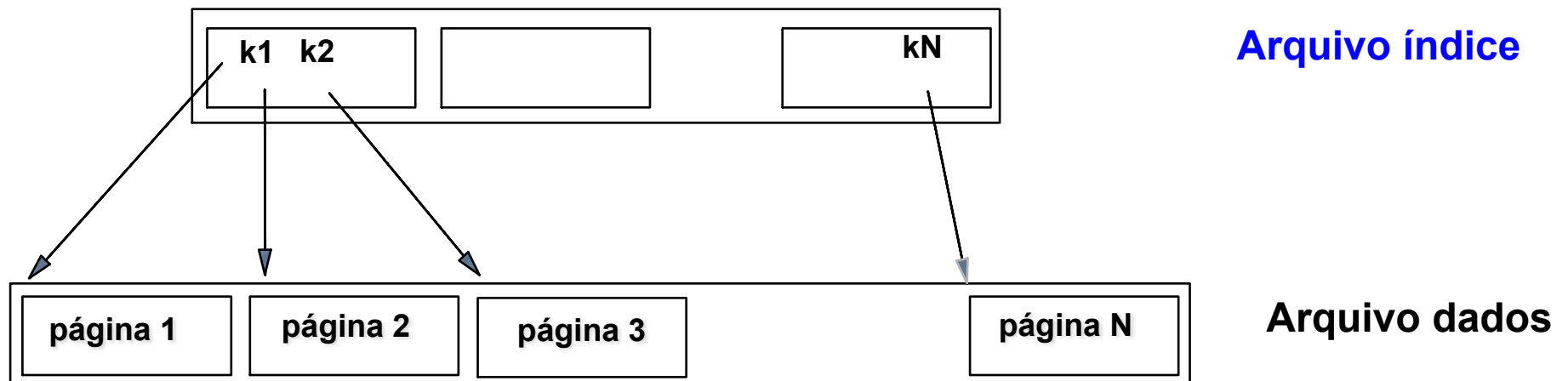
Javam Machado
DC - UFC

Introdução

- Arquivos Heap: Melhoria na recuperação dos dados
 - Busca por igualdade: predicado do tipo $At = valor$
 - Busca por intervalo: predicado do tipo $At \geq valor$
- Técnicas de indexação estruturadas em árvore suportam ambos *busca por intervalo* e *busca por igualdade*.
- ISAM: estrutura estática;
- Árvore B+: dinâmica, ajustes gradativos sob inserções e exclusões.

Busca por Intervalo

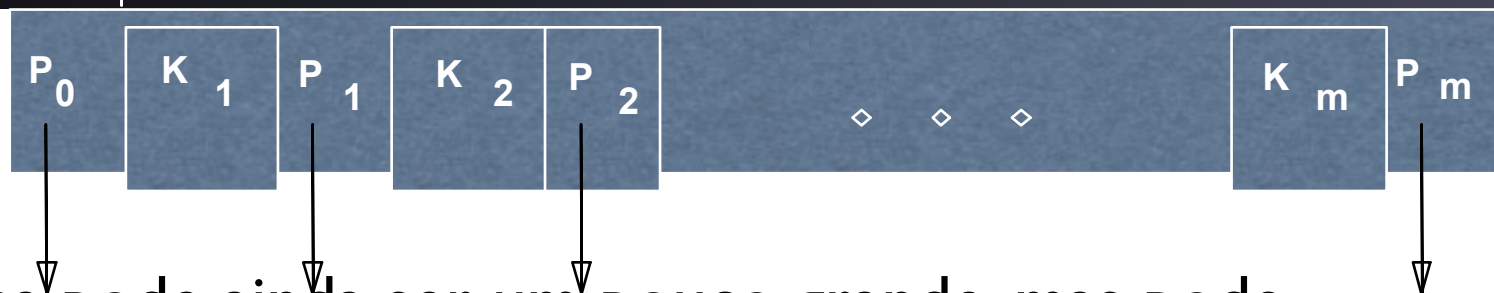
- “Encontre todos os estudantes com média > 3.0 ”
 - Se os dados estão em um arquivo ordenado, fazer busca binária para encontrar o primeiro estudante, então varrer para encontrar os outros.
 - Custo da busca binária pode ser um pouco alto.
- Idéia Simples: Criar um arquivo ‘índice’.



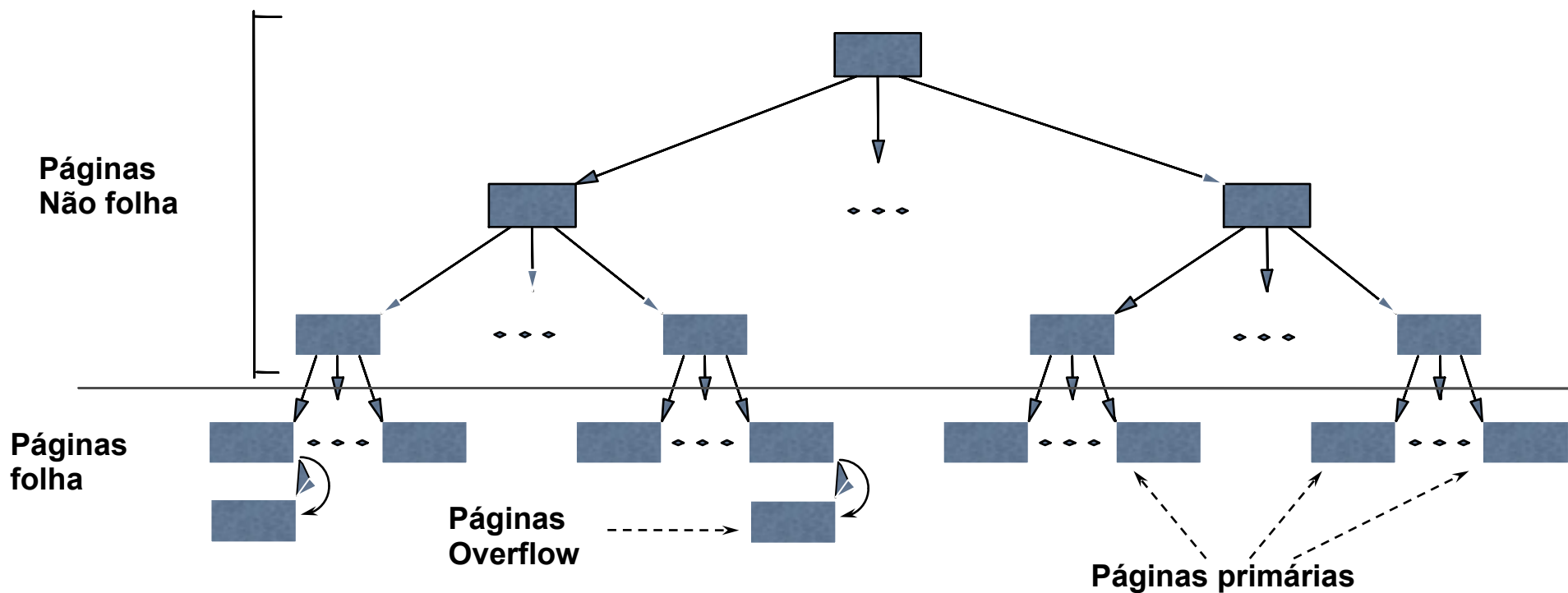
- Podemos fazer busca binária em um arquivo(menor) de índices!

ISAM

Entrada de índices



- O arquivo índice pode ainda ser um pouco grande, mas pode-se aplicar a idéia repetidamente!



- *Páginas folha contém entradas de dados*

Comentários sobre ISAM

- *Criação do Arquivo*: Entradas de dados alocadas seqüencialmente, ordenadas pela chave de busca; em seguida alocar página de índice e por fim o espaço para páginas de *overflow*.
- *Entrada do índice*: <valor da chave de busca, id da página>.
- *Busca*: Começa pela raiz; faz comparações de chave para ir para a folha. Custo $\log_f N$; $f = n^\circ$ de páginas de entradas/índices, $N = n^\circ$ páginas folha.
- *Inserção*: Achar a folha correspondente à entrada de dados e a atualizar.
- *Exclusão*: Achar e remover das folhas; se a página de overflow se torna vazia, liberá-la.
- *Árvore de Estrutura Estática*: inserções/exclusões afetam apenas páginas folha.

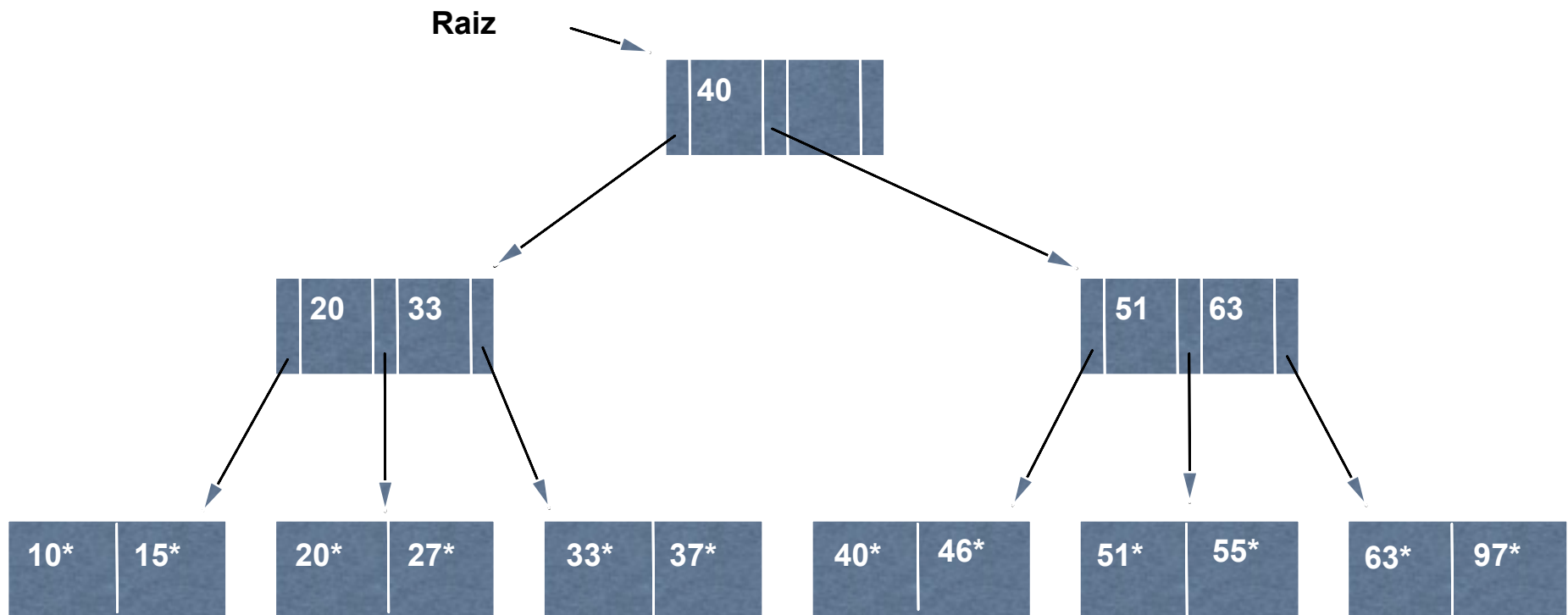
Páginas
de dados

Páginas
de índices

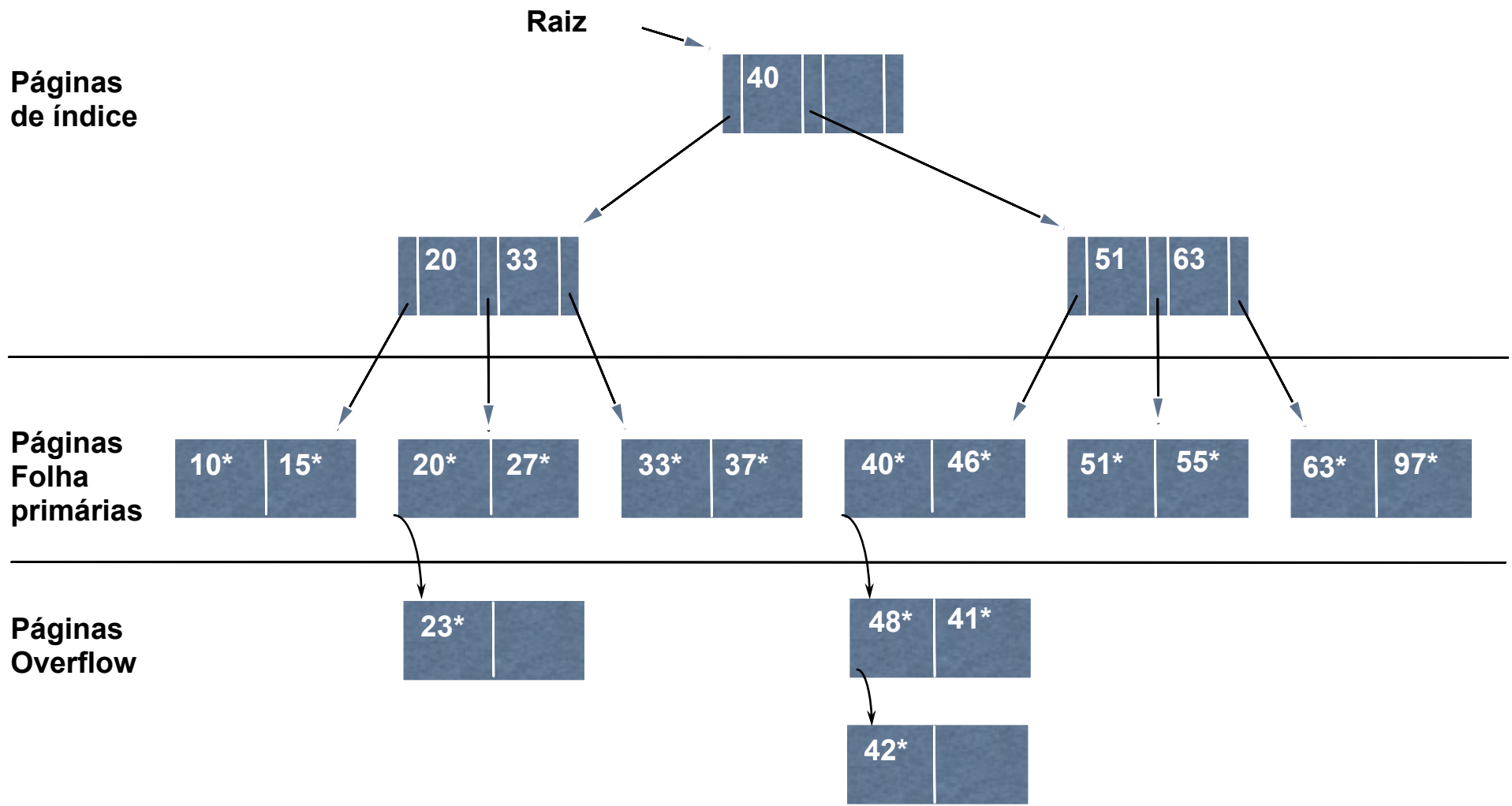
Páginas
Overflow

Exemplo de Árvore ISAM

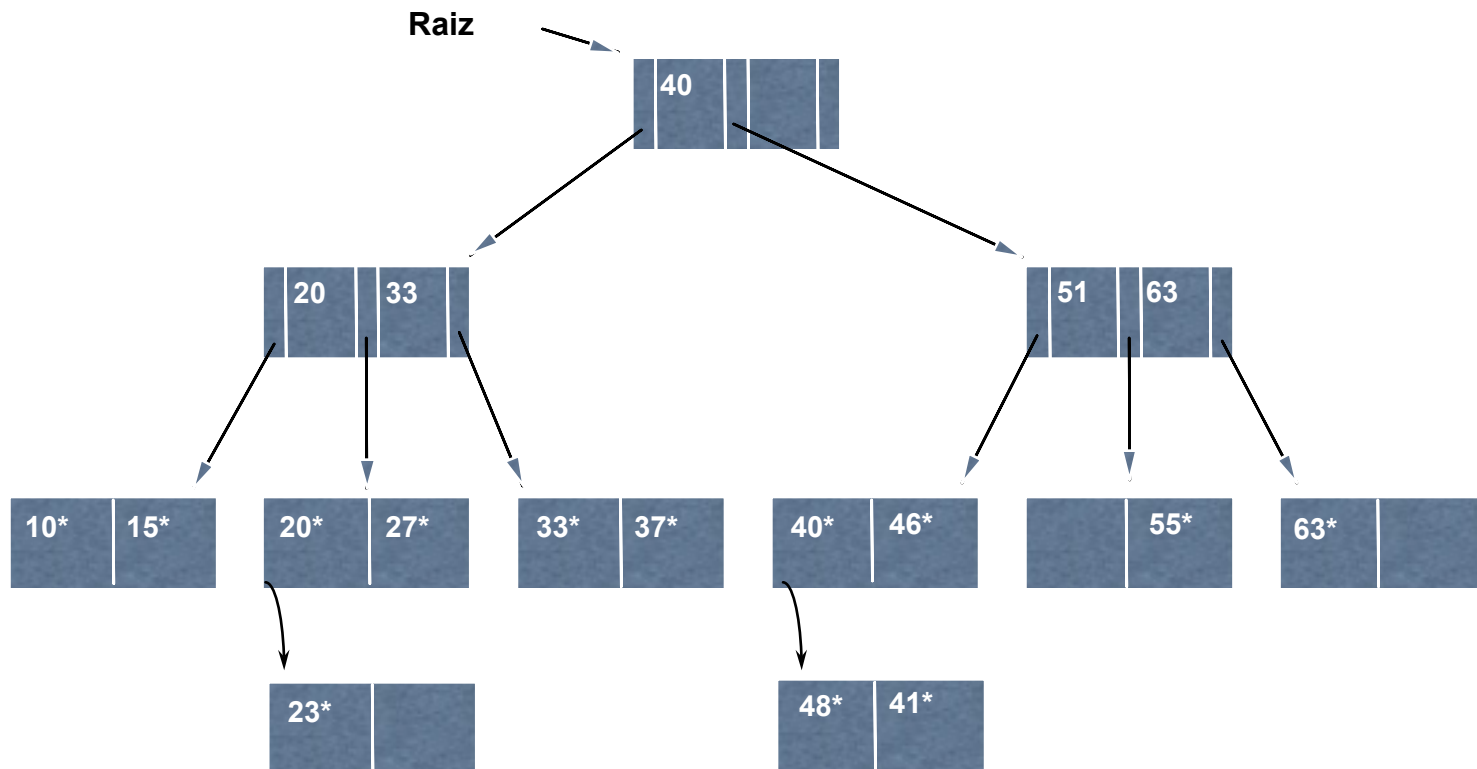
- Cada nó deve ter duas entradas; não precisa dos ponteiros 'próxima página folha'.



Após Inserir 23*, 48*, 41*, 42* ...



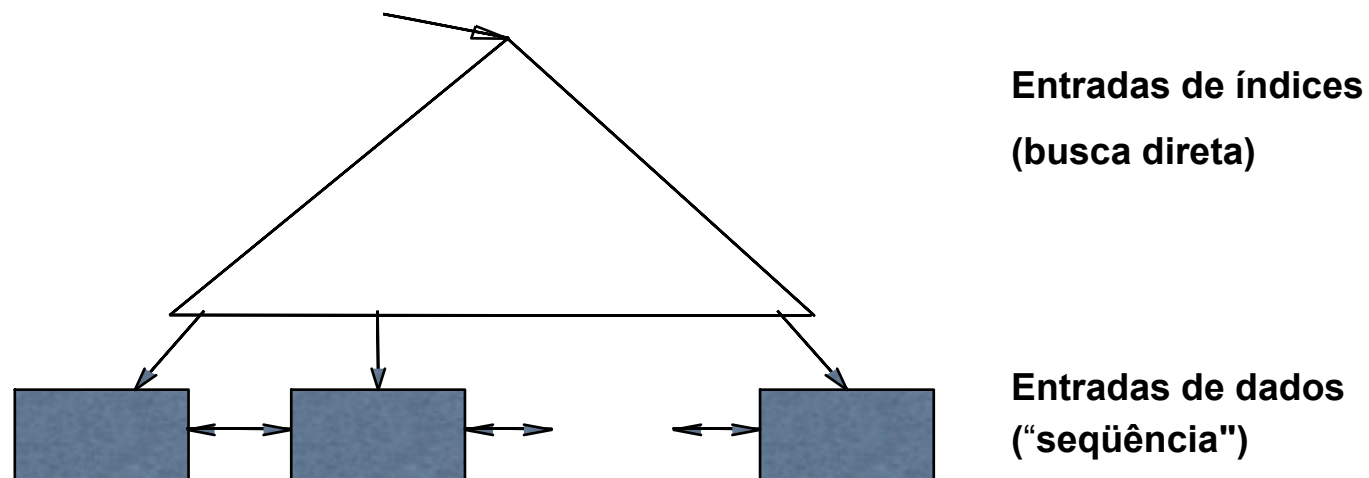
... Excluindo 42*, 51*, 97*



- Note que 51* aparece à nível de índice, mas não de folha!

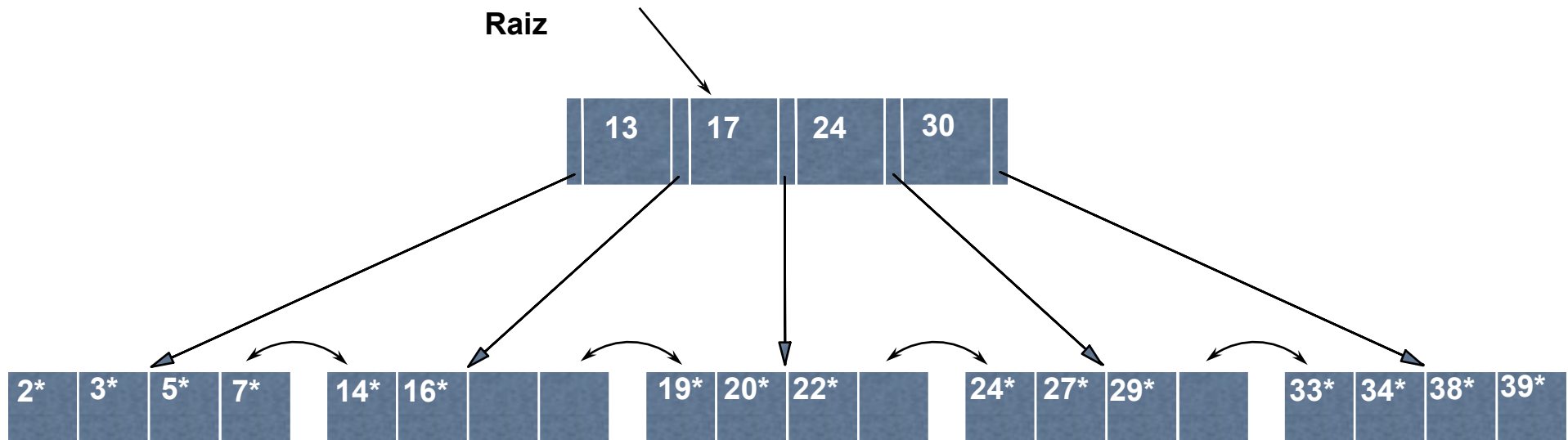
Árvores B+: mais usadas

- Inserção/exclusão com custo $\log_F N$; mantém a árvore com **altura balanceada**. (F = fanout, N = n° de páginas folha)
- Mínimo de 50% de ocupação (exceto para o nó raiz). Cada nó contém $d \leq \underline{m} \leq 2d$ entradas. O parâmetro d é chamado o ordenador da árvore.
- **Suporta busca por intervalo e por igualdade eficientemente.**



Exemplo de Árvore B+

- A busca começa na raiz, e a as comparações chave dirigem-no a uma folha (como em ISAM).
- Busca por 5*, 15*, todas as entradas de dados $\geq 24^*$...



- Com base na busca de 15*, sabemos que ele não está na árvore!

Árvores B+ na Prática

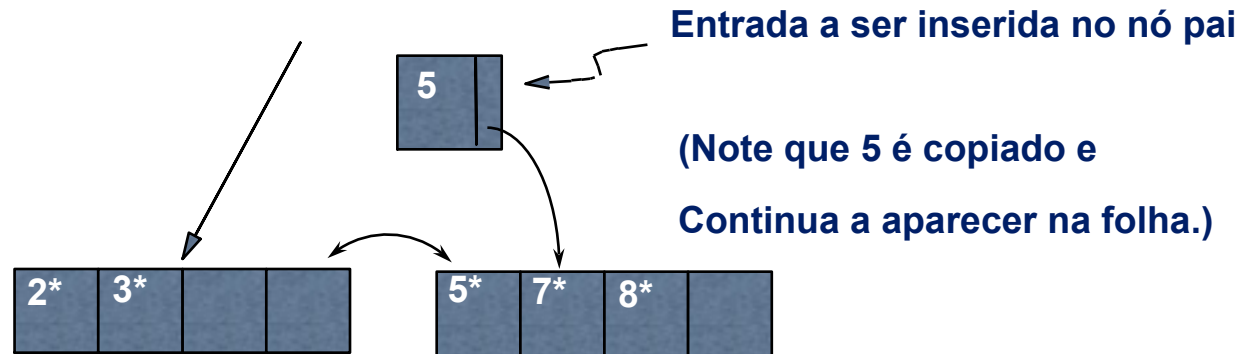
- Ordenação Típica: 100. Fator de preenchimento típico: 67%.
 - *Fanout* médio = 133
- Capacidades típicas:
 - Altura 4: $133^4 = 3\,129\,007\,700$ registros
 - Altura 3: $133^3 = 2\,352\,637$ registros
- Pode frequentemente manter níveis topo no *buffer*:
 - Nível 1 = 1 página = 8 Kbytes
 - Nível 2 = 133 páginas = 1 Mbyte
 - Nível 3 = 17.689 páginas = 133 MBytes

Inserindo uma Entrada de Dados em uma Árvore B+

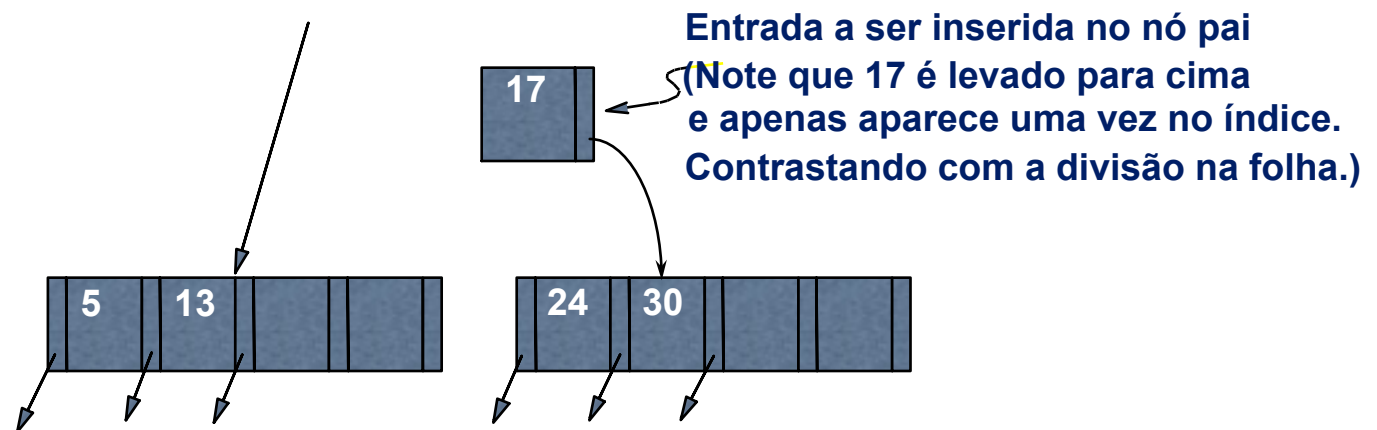
- Encontre a folha correta L .
- Ponha dados de entrada em L .
 - Se L tem espaço suficiente, *faça!*
 - Senão, você deve dividir (*split*) L (em L e um novo nó $L2$)
 - Redistribuir as entradas igualmente, subir a chave do centro.
 - Inserir entradas de índice apontando para $L2$ no pai de L .
- Isso pode ser feito recursivamente
 - Dividir o nó índice, redistribuir as entradas igualmente, mas subir a chave do centro. (Contrastando com a divisão das folhas.)
- Divisão (*split*) “aumenta” a árvore; a divisão da raiz aumenta a altura.
 - Aumento da árvore: pegue a largura ou um nível abaixo do topo.

Inserindo 8* na Árvore B+

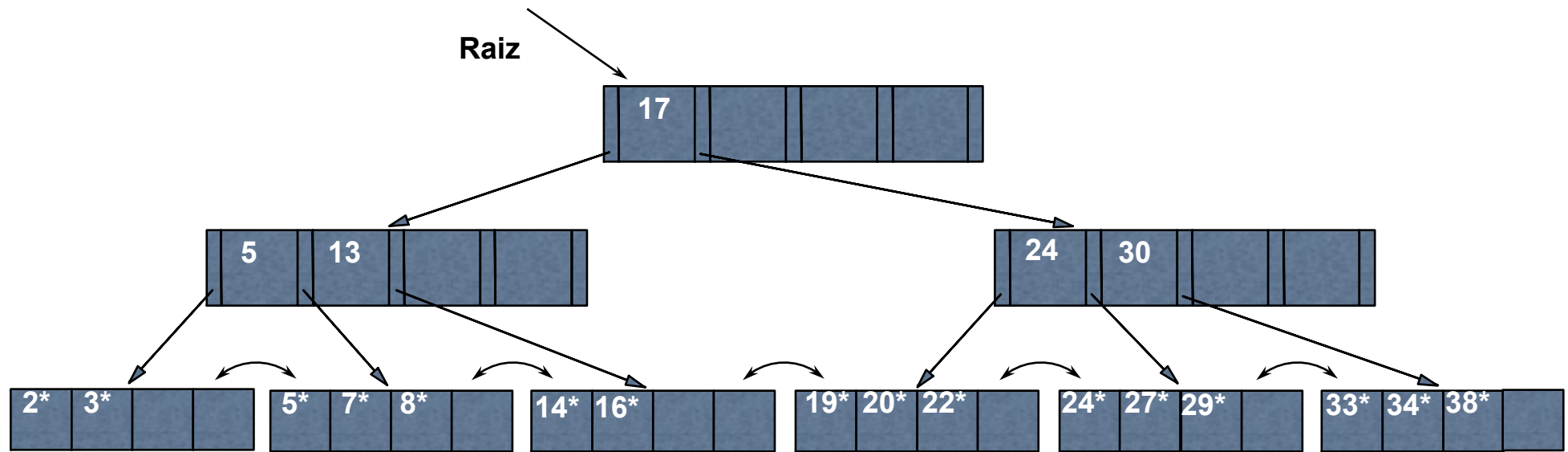
- Observe como a ocupação mínima é garantida em ambas: divisões de páginas folha e índice.



- Note a diferença entre copiar para cima e mover para cima.



Exemplo da Árvore B+ Após Inserir 8*



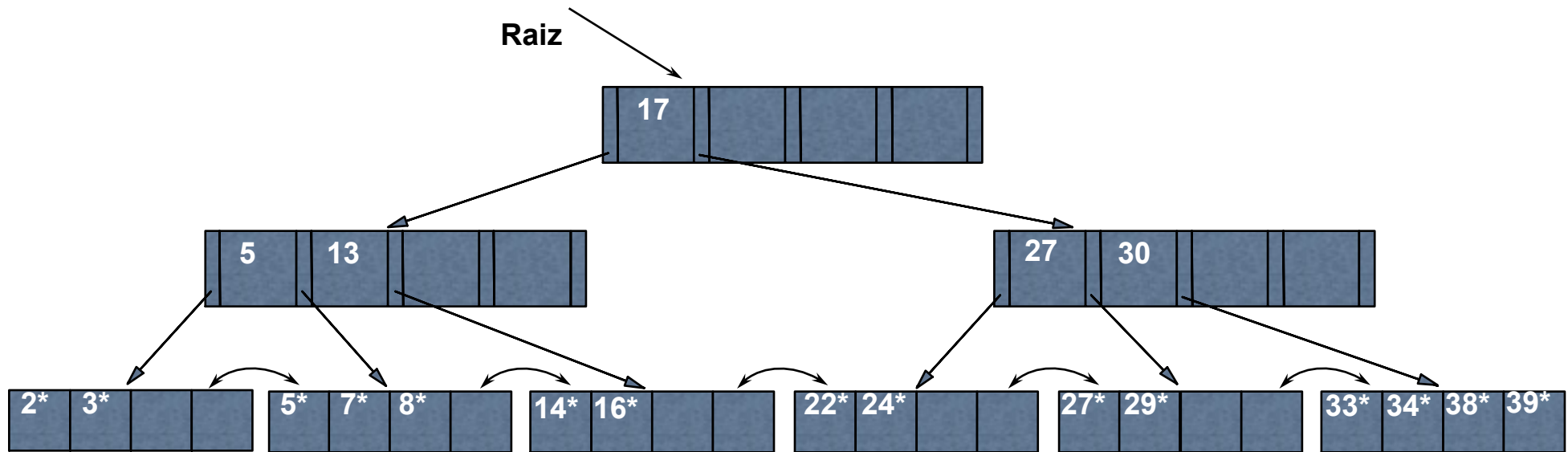
Observe que a raiz foi dividida, acarretando um aumento na altura.

Nesse exemplo, podemos evitar divisão pela re-distribuição das entradas; porém, isso não é geralmente feito na prática.

Excluindo uma Entrada de Dados de uma Árvore B+

- Comece na raiz, encontre a folha L que contém a entrada.
- Remova a entrada.
 - Se L está pelo menos com metade ocupada, *faça!*
 - Se L tem apenas **$d-1$** entradas,
 - Tente re-distribuir, pegando emprestado do irmão(sibling) (*nó adjacente com mesmo pai L*).
 - Se a re-distribuição falhar, faça um merge de L e irmão.
- Se houve *merge*, você deve excluir a entrada (apontando para L ou irmão) do pai de L .
- O *merge* pode ser propagado à raiz, diminuindo a altura.

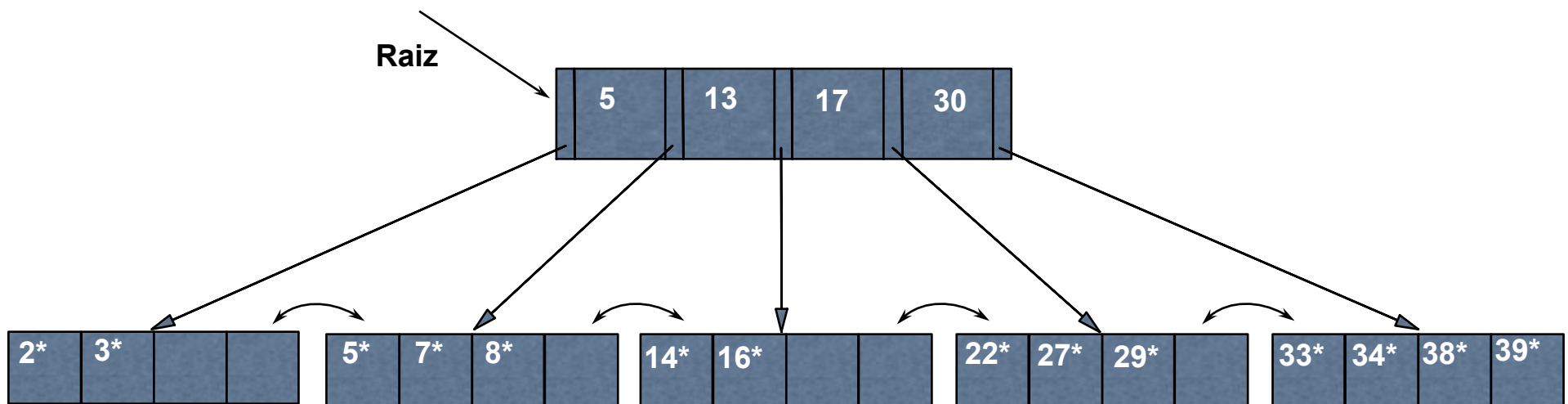
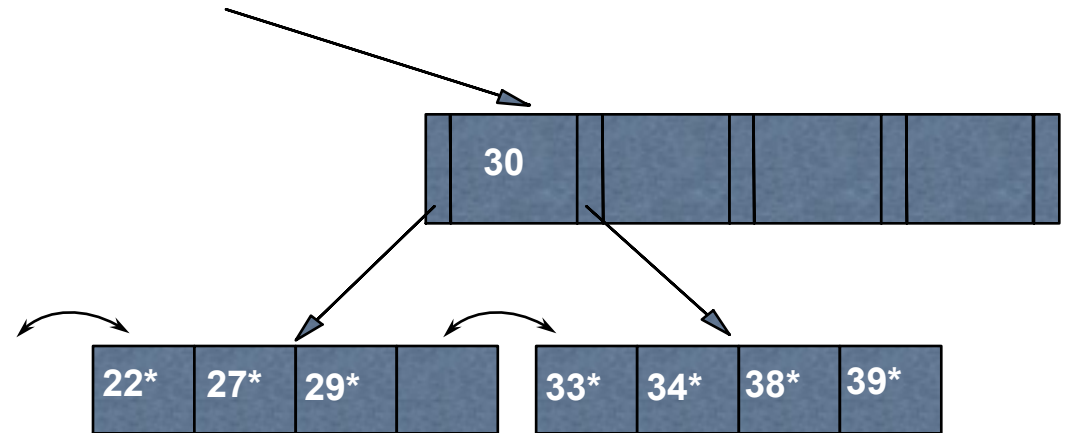
A Árvore após Excluir 19* e 20*



- Excluir 19* é fácil.
- Exclusão de 20* está feita com a re-distribuição. Observe como a chave central é copiada para cima.

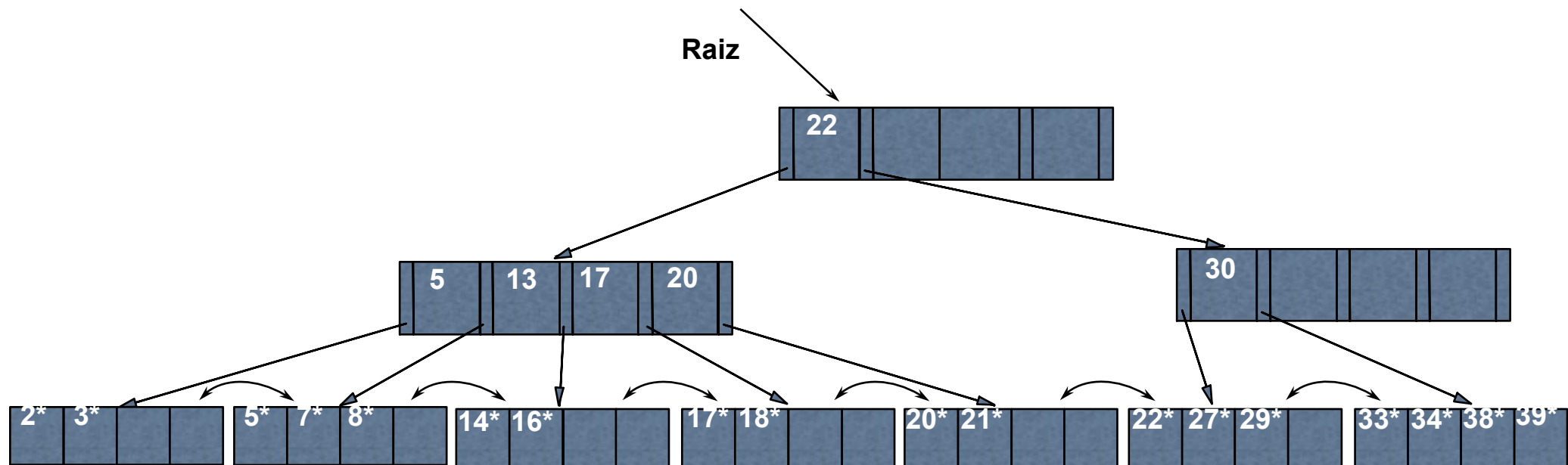
... e então Excluir 24*

- Você deve fazer o *merge*.
- Observe o “*carregamento*” da entrada de índice (à direita), e a decida da entrada de índice (abaixo).



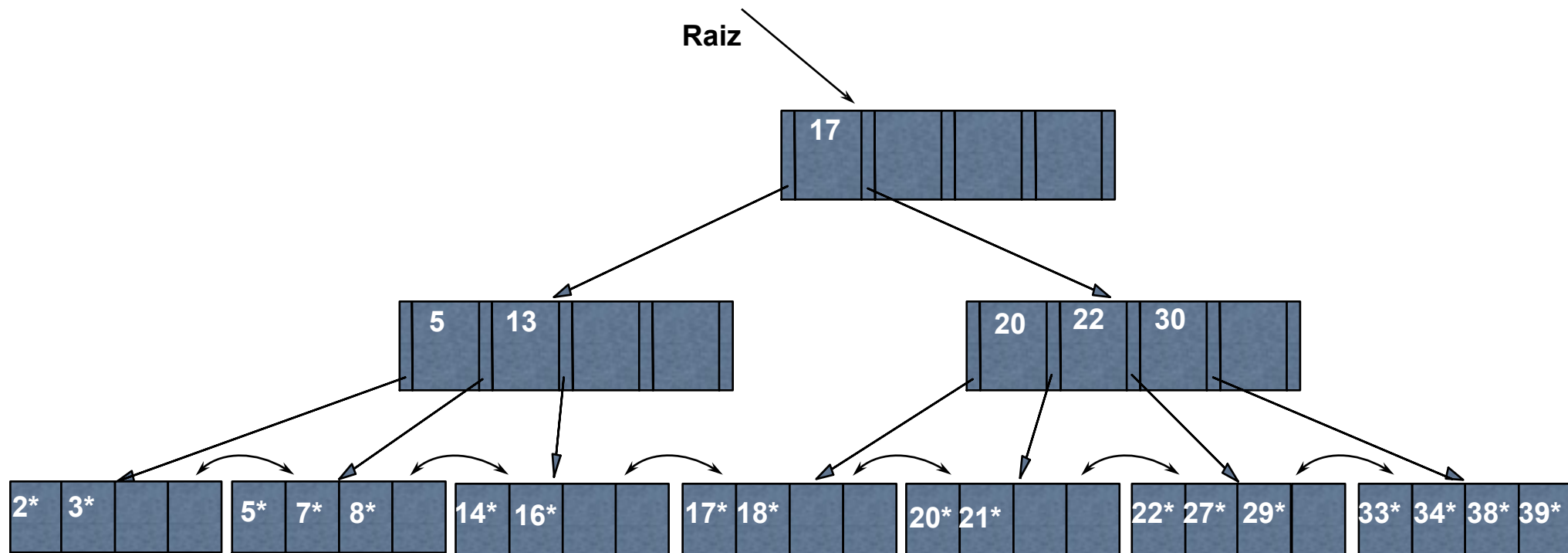
Exemplo de Redistribuição Interna

- A árvore é mostrada abaixo *durante a exclusão de 24**. (O que pode ser uma possível árvore inicial?)
- Em contraste ao exemplo anterior, podemos re-distribuir a entrada do filho direito para o esquerdo.



Após a Redistribuição

- Intuitivamente, entradas são re-distribuídas “empurrando completamente” a entrada de divisão no nó pai.
- É suficiente re-distribuir o índice de entrada com chave 20; tínhamos re-distribuído 17 como ilustração.

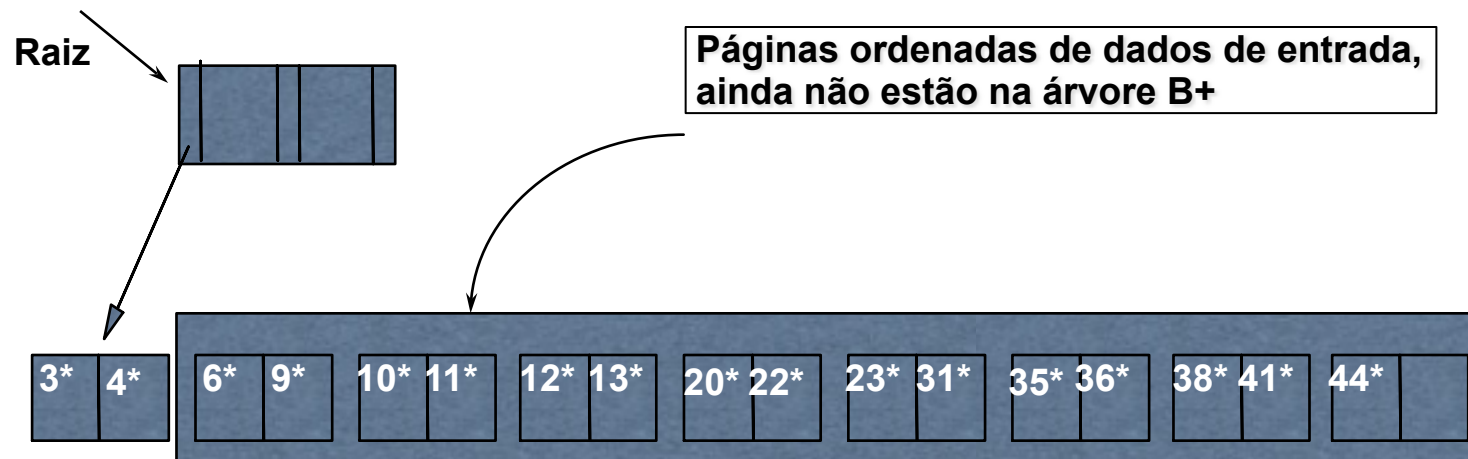


Prefix Key Compression

- Importante para incrementar o *fan-out*. (Por que?)
- Compressão do valores de chave nos índices.
 - E.g., Se tivermos índices de entrada adjacentes com valores de chave de busca *Paulo de Tarso*, *Pablo Picasso* e *Pedro Pereira*, podemos abreviar *Pablo Picasso* para *Pab*.
 - Isso é correto? Não muito! Se tivermos uma entrada de dados *Pablito Tevez*? (Podemos apenas comprimir *Pablo Picasso* para *Pablo*)
 - Em geral, durante a compressão, devemos levar cada entrada de índice maior que todo valor chave (em qualquer sub-árvore) para sua esquerda.
- Inserção/deleção devem ser apropriadamente modificados.

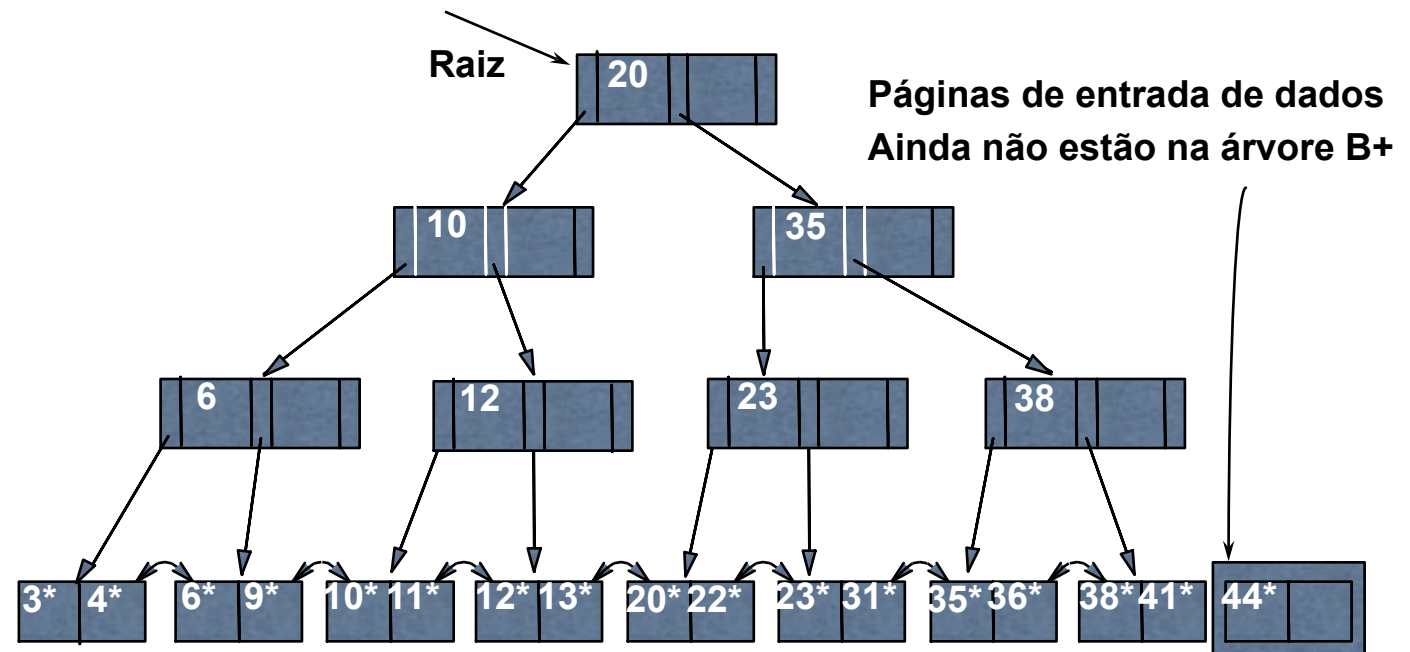
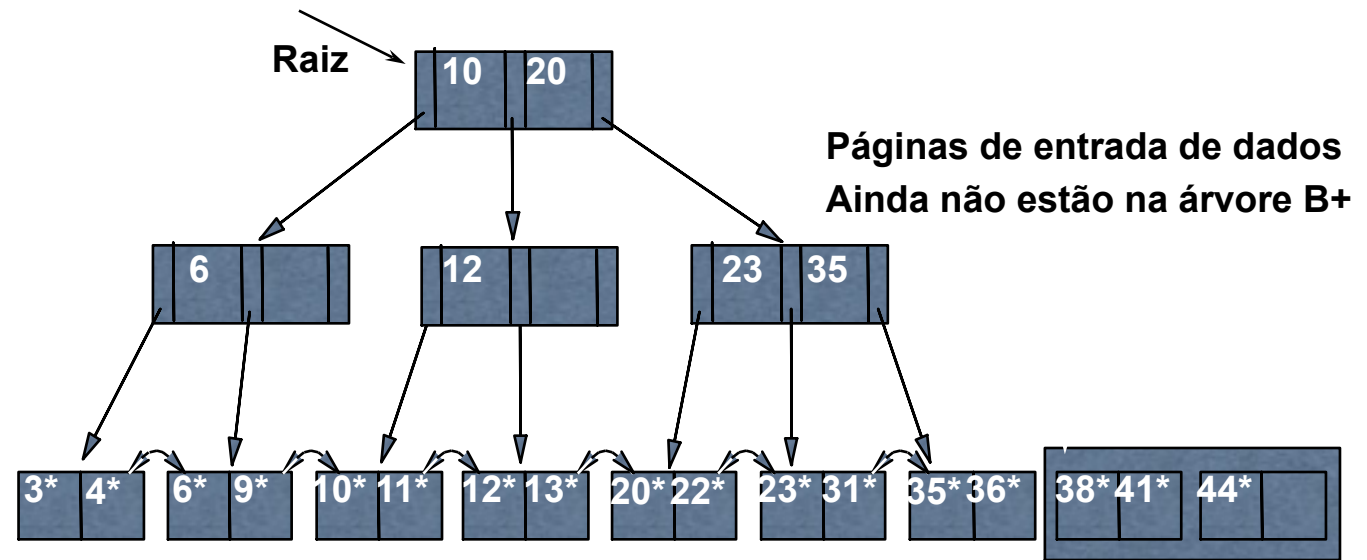
Bulk Loading de uma Árvore B+

- Se tivermos uma grande coleção de registros, e quisermos criar uma árvore B+ em algum campo, fazer isso pela inserção repetida dos registros é muito lento.
- Carregamento em Massa (Bulk Loading) pode ser feito mais eficientemente.
- *Inicialização*: Ordena todas as entradas de dados, insere ponteiro para a primeira(folha) página em uma nova página (raiz).



Bulk Loading (Cont.)

- Índices de entrada para páginas folha sempre foram introduzidas na página de índice mais à direita exatamente sobre o nível folha. Quando esse é preenchido, é dividido. (A divisão pode construir um caminho mais à direita para a raiz.)
- Muito mais rápido que inserções repetidas, especialmente quando considera bloqueio!



Sumário de *Bulk Loading*

- Opção 1: múltiplas inserções.
 - Lento.
 - Não faz armazenamento seqüencial das folhas.
- Opção 2: *Bulk Loading*
 - Tem vantagens para controle de concorrência.
 - Poucos I/Os durante o carregamento.
 - Folhas armazenadas seqüencialmente (e “linkadas”, é claro).
 - Pode controlar o “fator de preenchimento” nas páginas.

Sumário

- Índices estruturados em árvores são ideais para *range-searches*, também bons para *equality searches*.
- ISAM é uma estrutura estática.
 - Apenas páginas folhas são modificadas; páginas *overflow* são necessárias.
 - Correntes de *overflow* podem degradar o desempenho a menos que o tamanho do conjunto de dados e a distribuição de dados permaneça constante.
- Árvore B+ estrutura dinâmica.
 - Inserções/deleções deixam a árvore balanceada na altura; com custo $\log_F N$.
 - Alto *fanout* (**F**) significa que a profundidade raramente é maior que 3 ou 4.
 - Melhor que manutenção de um arquivo ordenado.

Sumário (Cont.)

- Tipicamente, **67%** de ocupação média.
- Usualmente preferível ao ISAM, considerando modo **bloqueio**; ajusta para aumentar gradativamente.
- Se as entradas de dados são registros, dividir(*split*) pode mudar os rids!
- Comprimir as chaves pode aumentar o *fanout*, reduz o tamanho.
- *Bulk loading* pode ser muito mais rápido que inserções repetidas criando uma árvore B+ num grande conjunto de dados.
- É o índice mais amplamente usado em SGBDs devido à sua versatilidade. É um dos componentes mais otimizados de um SGBD.